

Adaptive Start Algorithm for Thermostats

July 14, 2026 at 10:56 AM

Gemini Conversation 27f7df4d

2 conversation turns

Turn 1

User

Hi Gemini 剛才的對話你建議要做Adaptive Start的功能，但是這個功能不是很容易做。會因為環境溫度/室內外溫差/目標溫度的不同，影響計算的結果。你有建議的算法嗎

Assistant

確實，**自適應啟動 (Adaptive Start / Optimum Start)** 是溫控器開發裡公認的「硬骨頭」。特別是地暖系統 (Underfloor Heating)，其熱慣性巨大，升溫曲線高度非線性，還會受到建築保溫性能和室內外溫差 (熱損失率) 的強烈影響。

如果算法寫得太死，春天會過熱，冬天又熱不起來；如果搞得太複雜 (例如導入深度學習)，在資源有限的微控制器 (MCU) 上又跑不動。

為了在**運算資源與精準度**之間取得平衡，業界在嵌入式系統中主要使用以下幾種逐步演進的算法。我為你整理了三種實用的架構，從簡單到專業：

方案一：動態溫升率法 (Heuristic Adaptive Rate of Rise) —— 最易實現

這是最經典的商用溫控器算法。它不直接計算複雜的熱力學物理模型，而是把「**溫升率 (R)**」當作一個會自我修正的變數。

1. 核心公式

預熱所需時間 (分鐘) Δt 計算如下：

$$\Delta t = \frac{T_{\text{target}} - T_{\text{current}}}{R}$$

其中 R 為當前的**溫升率** ($^{\circ}\text{C}/\text{min}$)。

2. 自適應學習機制 (每日更新)

每天在預熱結束、達到目標溫度後，系統會計算「實際達到溫度的時間」與「預估時間」的誤差，並使用一階濾波 (IIR Filter) 來更新 R ：

- **實際溫升率：**

$$R_{\text{actual}} = \frac{T_{\text{target}} - T_{\text{start}}}{\text{Actual Heating Time}}$$

- **更新溫升率** (引入學習率 η ，通常取 $0.1 \sim 0.2$)：

$$R_{\text{new}} = (1 - \eta) \cdot R_{\text{old}} + \eta \cdot R_{\text{actual}}$$

💡 優點：極度節省 RAM/Flash，運算極簡單。 ⚠️ 缺點：沒有考慮室內外溫差。如果冬天到春天的氣溫劇烈變化，系統需要 3~5 天才能「學會」新的溫升率，期間會出現超溫或不夠熱的情況。

方案二：多變數線性迴歸與漸進學習法 (Multi-Variable Regression) —— 推薦首選

如果你有**室外溫度**（透過感測器或天氣 API）或想引入**室內起點溫度**作為熱損失的代理變數，這是最平衡且精準的作法。

這個算法預估預熱時間 Δt 的公式如下：

$$\Delta t = a \cdot \Delta T + b \cdot (T_{\text{current}} - T_{\text{outdoor}}) + c$$

- $\Delta T = T_{\text{target}} - T_{\text{current}}$ （溫差）
- $T_{\text{current}} - T_{\text{outdoor}}$ （代表目前的熱損失速度）
- a, b, c 是系統需要學習的係數。

學習更新算法（梯度下降簡化版）

每次加熱完成後，我們會得到一個實際的預熱時間 Δt_{actual} 。我們可以使用**最小平方法 (LMS/RLS)** 或簡單的**增量修正**來微調 a, b, c 。

例如，最直觀的誤差修正法：

$$e = \Delta t_{\text{actual}} - \Delta t_{\text{predict}}$$

$$a_{\text{new}} = a_{\text{old}} + \mu \cdot e \cdot \Delta T$$

$$b_{\text{new}} = b_{\text{old}} + \mu \cdot e \cdot (T_{\text{current}} - T_{\text{outdoor}})$$

（其中 μ 為一個極小的步長，例如 0.01）

方案三：熱阻電容模型 (RC Thermal Model) —— 物理學霸路線

如果你追求極致的控制品質，特別是地暖這種「熱容（Thermal Mass）」極大的系統，可以將房間與地板簡化為一階或二階 **RC 等效電路**。

$$\frac{dT}{dt} = \frac{1}{C} \left(P_{\text{heater}} - \frac{T_{\text{current}} - T_{\text{outdoor}}}{R_{\text{thermal}}} \right)$$

- C ：房間與地塊的熱容（蓄熱能力）。
- R_{thermal} ：房屋的熱阻（保溫能力）。
- P_{heater} ：地熱的加熱功率。

嵌入式實現步驟

1. **系統辨識（System Identification）**：在系統初次安裝或運行前幾天，全功率加熱，記錄溫升曲線，利用最小平方法估算出這棟房子的 R_{thermal} 和 C 。
2. **逆向求解時間**：當已知 R_{thermal} 、 C 、當前室溫 T_{current} 、室外溫 T_{outdoor} 與目標溫度 T_{target} 時，逆向積分求解從 T_{current} 爬升到 T_{target} 所需的時間 t ：

$$T(t) = T_{\text{outdoor}} + R \cdot P - (R \cdot P - (T_{\text{current}} - T_{\text{outdoor}})) e^{-\frac{t}{RC}}$$

解出 t 即為你的啟動延遲時間。

✂ 實務開發上的「避坑指南」

1. **限制最大預熱時間（Max Limit）** 地暖的預熱時間可能長達 2~4 小時。務必在軟體中設定一個**最大上限限制**（例如 4 小時）。不要讓算法因為某一次的極端低溫，計算出一個需要提早 10 小時啟動的荒謬結果。
2. **設定死區（Deadband）與防抖** 如果 $T_{\text{target}} - T_{\text{current}} < 0.5^{\circ}\text{C}$ ，直接停用自適應啟動，避免頻繁微幅加熱造成的繼電器損耗。
3. **非線性修正（升溫後半段變慢）** 地暖在接近目標溫度時，因為溫差變小，熱交換速度會變慢。如果使用方案一（線性法），建議將目標溫度設定一個補償值（例如 $T_{\text{target}} - 0.3^{\circ}\text{C}$ 作為計算終點），剩下的微調交給 PID 閉環控制。

如果是要在一般 MCU（如 STM32F103）上跑，**方案一**加上「季節溫差補償因子」（或是簡化版的**方案二**）是性價比最高、客戶體驗也最好的選擇。

你目前規劃的溫控器硬體上，有預留接室外溫度感測器，還是完全依賴室內溫度感應呢？這會決定哪一種算法最適合你。

Turn 2

User

我目前沒有室外溫度的資訊

Assistant

沒有室外感測器是室內溫控器最常見的狀況。在這種限制下，**方案一（動態溫升率法）**就是你的標準答案。

既然沒有室外溫度的資訊，我們就必須把「室內當前溫度」和「預熱開始時的室內溫度」本身，當作環境熱損失的間接指標。因為如果室外很冷，室內在不加熱時的起點溫度通常也會比較低，或者降溫速度較快。

為了讓方案一在沒有室外溫度時更精準、更具魯棒性，我建議你對算法進行以下**兩項關鍵優化**：

優化一：加入「起點溫度補償」（二維溫升率表）

純線性的溫升率在不同的起點溫度下會有落差（例如室內從 15°C 升到 20°C ，跟從 18°C 升到 20°C ，因為與牆壁外壁的溫差不同，升溫斜率其實是非線性的）。

你可以建立一個簡單的分段（**Binning**）學習機制，在 MCU 內部配置 2 到 3 個不同的溫升率變數，而不是全時段共用同一個 R 。例如：

- R_{cold} （**嚴寒模式**）：當啟動時室溫 $T_{\text{current}} < 16^{\circ}\text{C}$ 時使用。
- R_{mild} （**舒適模式**）：當啟動時室溫 $T_{\text{current}} \geq 16^{\circ}\text{C}$ 時使用。

每次加熱完成後，根據**啟動時的起點溫度**落在哪個區間，就去更新該區間的 R 值。這樣一來，算法就能自動適應「冬天」與「春秋兩季」不同的升溫特性。

優化二：引進 EN 50559 的安全與限制邏輯

針對地暖系統，歐洲標準（如 EN 50559）非常強調節能與避免地板過熱。在編寫自適應啟動時，缺乏外部溫度資訊容易導致算法在極端情況下「暴走」，因此硬體與軟體的保護限制必須做足：

1. **動態滑動視窗限制（Sliding Window Limit）**：設定自適應啟動的提早時間**絕對不能超過 3 小時**（或依據地暖功率設定）。如果算出來大於 3 小時，一律強制在 3 小時前啟動。
2. **單向漸進學習（防爆衝保護）**：在更新溫升率 R 的一階濾波公式中：

$$R_{\text{new}} = (1 - \eta) \cdot R_{\text{old}} + \eta \cdot R_{\text{actual}}$$

建議將學習率 η 設定得**不對稱**：

- 當發現**升溫變慢**（冬天來了， $R_{\text{actual}} < R_{\text{old}}$ ）時，讓 $\eta = 0.2$ （快速學習，提早啟動，確保使用者不會挨凍）。
- 當發現**升溫變快**（春天來了， $R_{\text{actual}} > R_{\text{old}}$ ）時，讓 $\eta = 0.05$ （慢速學習，避免因為某一天家裡剛好人多、熱源多，導致隔天突然大幅延後啟動而造成室冷）。

推薦的 MCU 偽代碼架構 (C 語言實作參考)

這是一個適合在 MCU 裡跑的定時任務（例如每分鐘執行一次），用來判斷是否該切換到加熱狀態：

```
typedef struct {
    float r_mild;           // 溫升率：適合輕度寒冷 (e.g., >16C)
    float r_cold;           // 溫升率：適合高度寒冷 (e.g., <=16C)
    float learning_rate_up; // 升溫變快的學習率 (較小)
    float learning_rate_down; // 升溫變慢的學習率 (較大)
} AdaptiveStart_t;

// 檢查是否需要啟動自適應加熱
bool Check_Adaptive_Start(float t_current, float t_target, uint32_t
minutes_to_schedule, AdaptiveStart_t *settings) {
    if (t_current >= t_target - 0.2f) return false; // 接近或已達標，不啟動

    // 1. 根據目前室溫選擇適合的溫升率
    float current_R = (t_current <= 16.0f) ? settings->r_cold : settings->r_mild;

    // 2. 計算預估需要的預熱時間 (分鐘)
    float required_heating_time = (t_target - t_current) / current_R;

    // 3. 安全邊界限制 (最大不超過 180 分鐘)
    if (required_heating_time > 180.0f) {
        required_heating_time = 180.0f;
    }

    // 4. 如果距離排程設定的時間(分鐘)小於或等於預估加熱時間，觸發加熱
    if (minutes_to_schedule <= (uint32_t)required_heating_time) {
        return true;
    }

    return false;
}
```

採用這種「分段拉表 + 不對稱一階濾波」的作法，即使沒有室外溫度感測器，系統在運行 3 到 5 天後也能非常精準地抓到該房間的地暖熱慣性。

在你的系統設計中，這個自適應啟動的功能，是打算由使用者可以在 UI 上自行開啟或關閉 (ON/OFF) 的，還是作為系統底層的預設最佳化邏輯呢？

Exported from [Voyager](https://github.com/Nagi-ovo/gemini-voyager) (<https://github.com/Nagi-ovo/gemini-voyager>) • 2 conversation turns

Generated on July 14, 2026 at 10:56 AM